

软件业务场景功耗调节指南

EIC7700 系列 AI 数字 SoC

Rev 0.5
2024-10-24

声明

版权所有©2024，北京奕斯伟计算技术股份有限公司（以下简称“ESWIN”），保留所有权利。

ESWIN 在此产品中保留所有知识产权和专有权利。未经 ESWIN 授权，不得全部或部分发布、复制、分发、转载、修改、改编、翻译或制作本产品的衍生作品。

ESWIN 保留随时更新本文档或改进其中所提及的产品或服务的权利，而无需承担任何通知义务，且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

ESWIN 明确声明，不承担因应用或使用本产品而产生的任何责任，包括但不限于直接、间接、特殊、偶然、附带、惩戒性或后果性的、因使用本产品或文件所产生的损害赔偿赔偿责任，即使已知这种损害发生的可能性，亦概不负责。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标。

本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

修订记录

文档版本	日期	说明
V0.5	2024/10/24	首次发布

目 录

软件业务场景功耗调节指南 I

EIC7700 系列 AI 数字 SOC..... I

声明 I

目 录 III

图目录 V

1. 软件业务场景功耗调节概述..... 1

 1.1 概述 1

 1.2 软件调节方法概述 1

 1.2.1 温度监测 1

 1.2.2 驱动层软件调节 1

2. 温度监测方法..... 2

3. 驱动层软件调节方法..... 2

 3.1 DEVFREQ 2

 3.2 CPUFREQ..... 3

 3.3 ES_CLK 4

表目录

Table 3-1 EIC770x 上的 DFS 节点 2

Table 3-2 es_clk 功能 4

图目录

Figure 1-1 业务场景下功耗调节示意 1

Figure 2-1 读取芯片温度信息 2

Figure 3-1 cpupower 查询 cpu 频率 4

Figure 3-2 cpupower 设置 cpu 频率 4

1. 软件业务场景功耗调节概述

1.1 概述

在某些使用场景中，芯片可能因高功耗而导致发热量迅速上升。如果散热条件不足，芯片温度可能持续升高，导致系统强制重启或性能异常，严重情况下甚至损害芯片。为防止这种情况发生，通常采用动态功耗管理策略。当芯片温度超出设定的安全阈值时，通过适当降低系统性能（如降低处理器频率、减少运行线程或限制数据吞吐量）来减少功耗，从而抑制热量进一步积累，使芯片温度逐渐恢复到安全范围。待温度回落后，再将系统性能逐步恢复到原始状态。

例如，在高性能计算任务或复杂图形渲染中，如果芯片温度接近临界点，系统可以自动降低 CPU 或 GPU 的运行频率以控制功耗；在网络设备中，当散热条件受限时，可以暂时限制数据传输速率以减少热量产生。这种基于温度反馈的功耗调节机制，有助于提高设备的运行稳定性，避免因过热导致的频繁宕机或性能下降，从而延长设备的使用寿命并保障其在高负载条件下的可靠运行。

本文主要描述在本芯片平台业务场景下功耗调节的方法和需注意的事项。

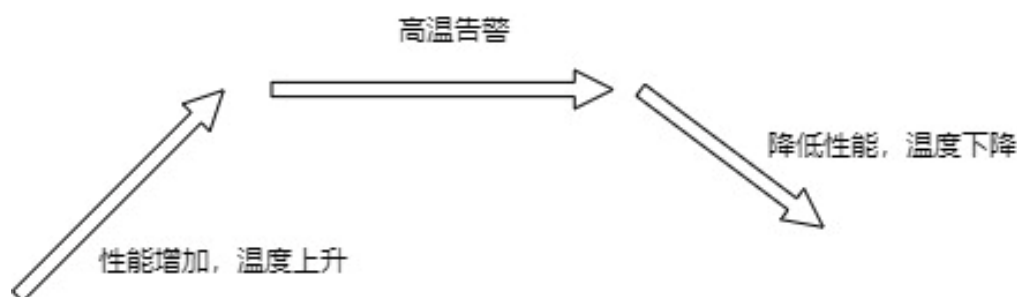


Figure 1-1 业务场景下功耗调节示意

1.2 软件调节方法概述

ESWIN 平台业务场景功耗调节主要包括了温度监测、驱动层软件功耗调节等几个方面。

1.2.1 温度监测

温度监测的目的是了解芯片当前的工作温度，判断是否超出了安全的温度范围。芯片内部带有 PVT（Process-Voltage-Temperature）模块，可以实时获取芯片的温度状态。

1.2.2 驱动层软件调节

通过改变各模块工作频率来改变系统功耗。

2. 温度监测方法

芯片的 PVT 驱动将温度信息通过 sys 节点的方式提供给了用户层查询。首先通过 sys 节点找到 label 为 pvt0 的 hwmon 节点，然后读取该节点下对应的温度输出。如下图所示：

```
eswin@rockos-eswin:~$ cat /sys/class/hwmon/hwmon0/label
pvt0
eswin@rockos-eswin:~$ cat /sys/class/hwmon/hwmon0/
device/          in2_input        name             templ_offset
in0_input        in2_label        of_node/         templ_type
in0_label        in3_input        subsystem/       uevent
in1_input        in3_label        templ_input      update_interval
in1_label        label            templ_label
eswin@rockos-eswin:~$ cat /sys/class/hwmon/hwmon0/templ_input
31872
eswin@rockos-eswin:~$
```

Figure 2-1 读取芯片温度信息

其中温度数值为毫摄氏度，如图中为 31872 m°C，也就是 31.872°C。

3. 驱动层软件调节方法

3.1 Devfreq

复杂 SoC 由多个子模块协同工作组成。在执行各种用例的操作系统中，并非 SoC 中的所有模块都需要始终保持最高性能。为方便起见，将 SoC 中的子模块分组为域，从而允许某些域以较低的电压和频率运行，而其他域以较高的电压/频率对运行，对于这些设备支持的频率和电压对，称之为 OPP (Operating Performance Point)。对于具有 OPP 功能的非 CPU 设备，称之为 OPP device，需要通过 devfreq 进行动态的调频调压。devfreq 则支持多个设备，并且允许每个设备有自己对应的 governor。Linux 内核实现了完整的 devfreq 框架，Eswin 平台上采用 userspace governor，在应用层创建了相应的文件节点来进行调频。

驱动层面的场景功耗调节，主要通过修改业务相关模块的工作频率来实现。基于 linux 的 devfreq 框架。各模块为应用层开辟了相应的功耗调节的文件节点。

Table 3-1 EIC770x 上的 DFS 节点

模块名称	DFS 节点
DSP0	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@0/
DSP1	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@1/

DSP2	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@2/
DSP3	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@3/
VDEC	/sys/class/devfreq/soc:video-decoder0@50100000/
VENC	/sys/class/devfreq/soc:video-encoder@50110000/
DEWARP	/sys/class/devfreq/51020000.dewarp/
HAE	/sys/class/devfreq/50140000.g2d/
NPU	/sys/class/devfreq/51c00000.eswin-npu/

通过 `echo xxx(期望的频率值) > /sys/class/devfreq/xxx/userspace/set_freq` 的方式，可以设置各模块的频点。

通过 `cat /sys/class/devfreq/xxx/available_frequencies` 的方式，可获取各模块支持的频点。

通过 `cat /sys/class/devfreq/xxx/cur_freq` 的方式，可获取各模块当前的频点。

3.2 CpuFreq

针对 CPU，本芯片平台上采用 linux CpuFreq 框架进行调频，在应用层生成了用于调频的文件节点，应用层可通过 `cpupower` 工具进行相应 CPU 的频率的设置。

`cpupower -c all frequency-info` 查看 CPU 当前的频率信息，包括：

- CPU 当前的工作频率；
- CPU 支持可调的工作频率列表；
- CPU 当前所使用的 governor；
- CPU 支持可选的 governor；

如下图所示：

```
[root@eswin-os:~]$ cpupower -c all frequency-info
analyzing CPU 0:
  driver: cpufreq-dt
  CPUs which run at the same hardware frequency: 0 1 2 3
  CPUs which need to have their frequency coordinated by software: 0 1 2 3
  maximum transition latency: 70.0 us
  hardware limits: 24.0 MHz - 1.80 GHz
  available frequency steps: 24.0 MHz, 100.0 MHz, 200 MHz, 400 MHz, 500 MHz, 600 MHz, 700 MHz, 800 MHz, 900 MHz, 1000 MHz, 1.20 GHz, 1.30 GHz, 1.40 GHz, 1.60 GHz, 1.80 GHz
  available cpufreq governors: conservative ondemand userspace powersave performance schedutil
  current policy: frequency should be within 24.0 MHz and 1.80 GHz.
                   The governor "userspace" may decide which speed to use
                   within this range.
  current CPU frequency: 1.40 GHz (asserted by call to hardware)
analyzing CPU 1:
  driver: cpufreq-dt
  CPUs which run at the same hardware frequency: 0 1 2 3
  CPUs which need to have their frequency coordinated by software: 0 1 2 3
  maximum transition latency: 70.0 us
  hardware limits: 24.0 MHz - 1.80 GHz
  available frequency steps: 24.0 MHz, 100.0 MHz, 200 MHz, 400 MHz, 500 MHz, 600 MHz, 700 MHz, 800 MHz, 900 MHz, 1000 MHz, 1.20 GHz, 1.30 GHz, 1.40 GHz, 1.60 GHz, 1.80 GHz
  available cpufreq governors: conservative ondemand userspace powersave performance schedutil
  current policy: frequency should be within 24.0 MHz and 1.80 GHz.
                   The governor "userspace" may decide which speed to use
                   within this range.
  current CPU frequency: 1.40 GHz (asserted by call to hardware)
analyzing CPU 2:
  driver: cpufreq-dt
  CPUs which run at the same hardware frequency: 0 1 2 3
  CPUs which need to have their frequency coordinated by software: 0 1 2 3
  maximum transition latency: 70.0 us
  hardware limits: 24.0 MHz - 1.80 GHz
  available frequency steps: 24.0 MHz, 100.0 MHz, 200 MHz, 400 MHz, 500 MHz, 600 MHz, 700 MHz, 800 MHz, 900 MHz, 1000 MHz, 1.20 GHz, 1.30 GHz, 1.40 GHz, 1.60 GHz, 1.80 GHz
  available cpufreq governors: conservative ondemand userspace powersave performance schedutil
  current policy: frequency should be within 24.0 MHz and 1.80 GHz.
                   The governor "userspace" may decide which speed to use
                   within this range.
  current CPU frequency: 1.40 GHz (asserted by call to hardware)
analyzing CPU 3:
  driver: cpufreq-dt
  CPUs which run at the same hardware frequency: 0 1 2 3
  CPUs which need to have their frequency coordinated by software: 0 1 2 3
  maximum transition latency: 70.0 us
```

Figure 3-1 cpupower 查询 CPU 频率

cpupower --cpu all frequency-set --freq xxxMHz 设置需要的 cpu 频率，如下图所示：

```
[root@eswin-os:~]$ cpupower --cpu all frequency-set --freq 1400MHz
Setting cpu: 0
Setting cpu: 1
Setting cpu: 2
Setting cpu: 3
[root@eswin-os:~]$
```

Figure 3-2 cpupower 设置 CPU 频率

需要注意的是，每个 DIE 的 CPU CORE 共享一套频率组合，频率的调节也是本 DIE 上对所有 CPU CORE 同时生效的。

3.3 es_clk

为方便应用层配置上述频点，我们在/usr/bin/下提供了 es_clk 脚本，可以用来快速设置 Devfreq 频点，可参考脚本修改添加需要的频点，脚本使用方法如下：

Table 3-2 es_clk 功能

输入	功能
----	----

es_clk -g <module>	查询指定模块 clk 模块名称(大写): VENC / VDEC / DSP0 / DSP1 / DSP2 / DSP3 / DEWARP / HAE / NPU 。 如未指定 module，则查询所有模块。
es_clk -s <module> <clk>	设置指定模块 clk 模块名称(大写): VENC / VDEC / DSP0 / DSP1 / DSP2 / DSP3 / DEWARP / HAE / NPU
es_clk -q <module>	查询指定模块所支持的 clk 频点 模块名称(大写): VENC / VDEC / DSP0 / DSP1 / DSP2 / DSP3 / DEWARP / HAE / NPU 如未指定 module，则查询所有模块。